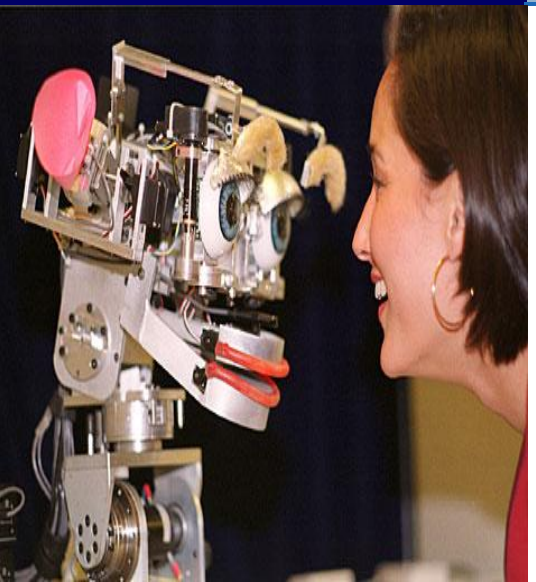
BIT.MLMR

Nouvelle AI

Lecture 8






Robots of Brooks in MIT

- **Intelligence without Representation and Reason**



Cog: *Humanoid intelligence requires humanoid interactions with the world.*



Genghis: *I used to live in the Mobot Lab. Now I live at the Smithsonian Air and Space Museum.*



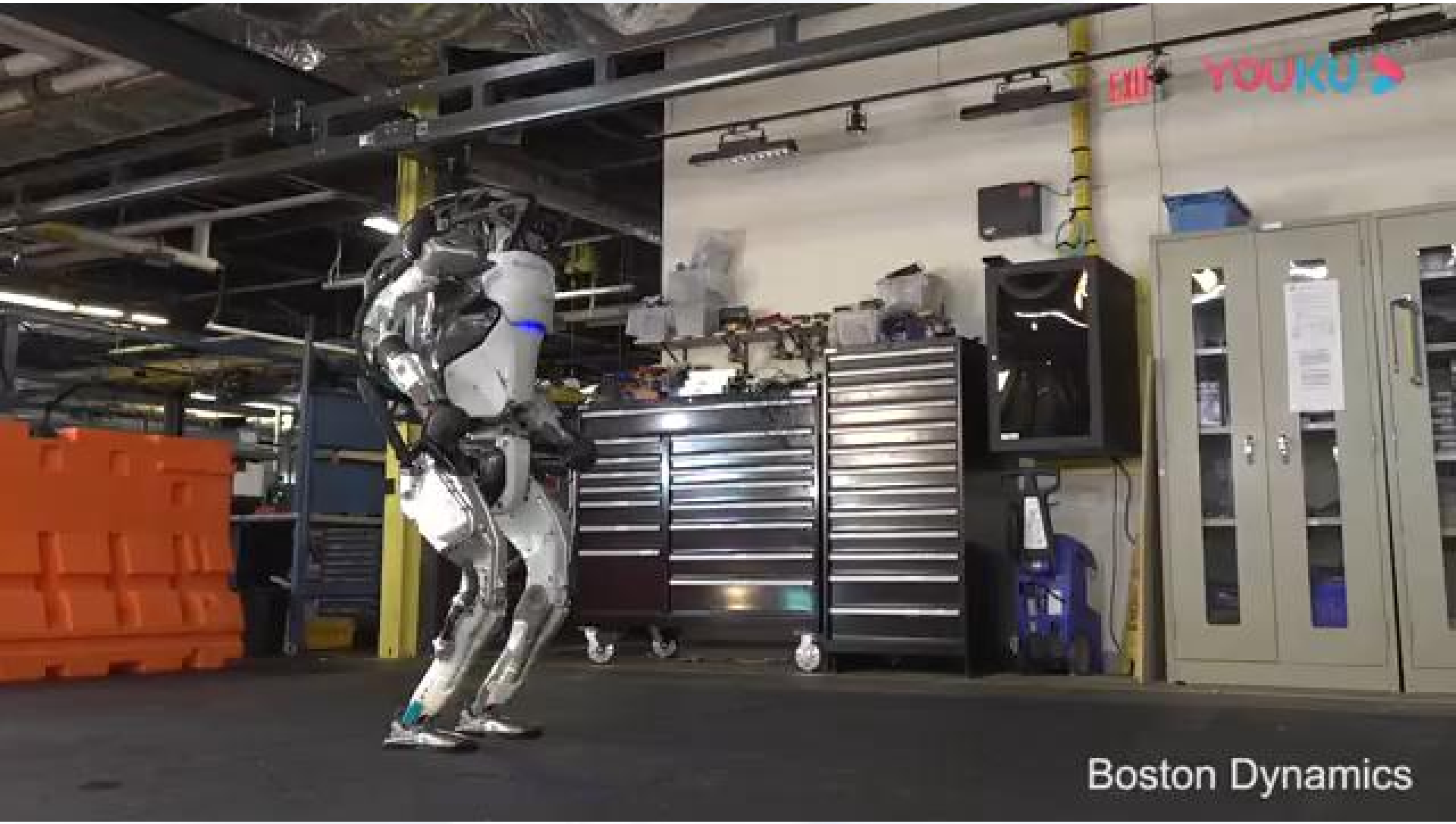
Herbert: *A reactive-based robot that collects soda cans, sometimes.*



Allen: *The Mobot Lab's first mobile robot.*

http://www.ai.mit.edu/projects/humanoid-robotics-group/

BostonDynamics Robot



Tic-Tac-Toe Robot



Outline

- **Autonomous Agents**
- **Reinforcement Learning**
  - **Q-Learning**
  - **TD(lamda)-Learning**
  - **REINFORCE**

Part I : Autonomous Agents

Agent Definitions

- **Russell and Norvig:**

  "anything that can be viewed as perceiving its environment through sensors and acting upon that environment through effectors"

- **Franklin and Graesser:**

  "An autonomous agent ... senses that environment and acts on it, over time, in pursuit of its own agenda and so as to affect what it senses in the future."

Weak Notion of Agents

- **Five key qualities:**
  - *Situatedness*: situated in the world, not in disembodied systems.
  - *Autonomous*: function without intervention
  - *Proactive*: goal-directed behaviour
  - *Reactive*: perceive and respond to changing environment
  - *Social ability*: interaction with others

Strong notion of agents

- In addition to the weak notion, also uses *mental components* such as
  - Belief
  - Desire
  - Intention
  - knowledge
  - etc

BDI Model

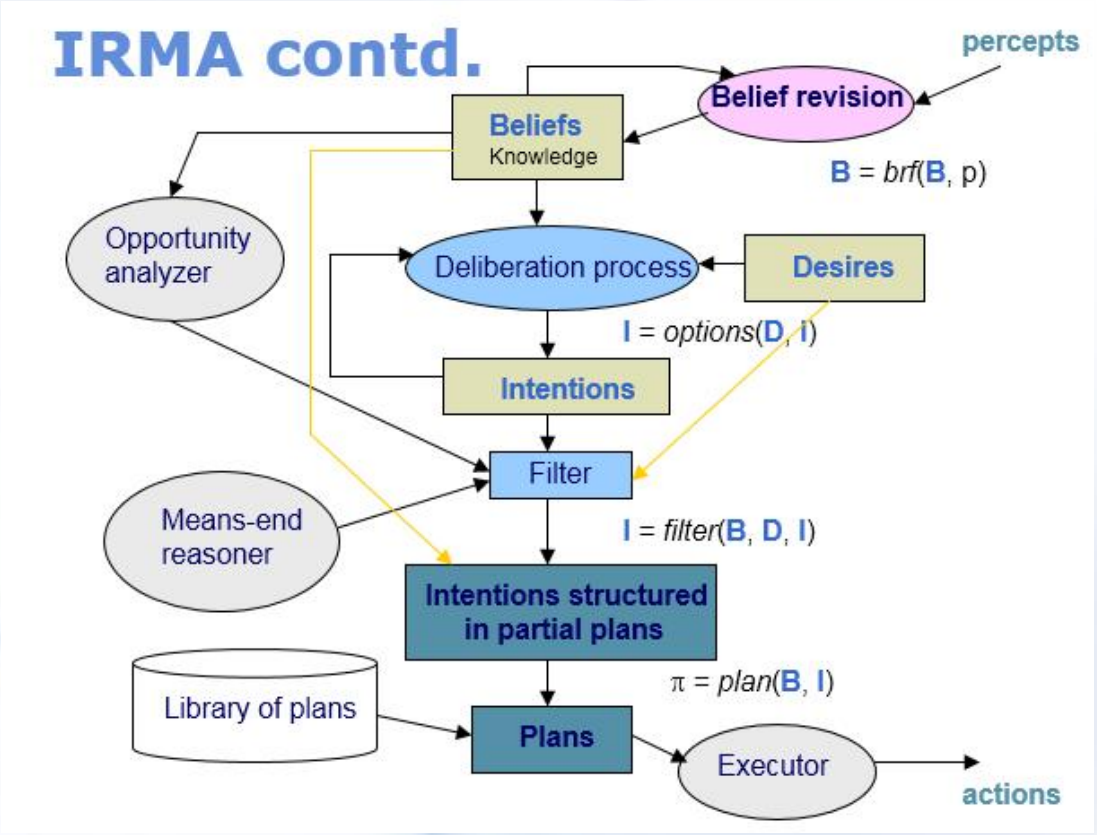
- BDI aims to model rational or intentional agency
- The symbols representing the world correspond to mental attitudes.
- Three categories:
  - *informative* (knowledge, Belief, assumptions)
  - *motivational* (Desires, motivations, goals)
  - *deliberative* (Intentions, plans)

Folk Psychology

- **E.g.,**
  - I *believed* the tutorial today was at 8:30am so I *intended* to arrive yesterday from London.
  - I *believed* the planes were not delayed and *desired* not to be late so I *intended* to arrive by 6pm.
- **Compelling because**
  - familiar: what it wants, knows and intends - easier to understand and predict behaviour.
  - Other agents can understand and predict behaviours
  - Relationship between these three categories may give us a handle on intelligent action in general.

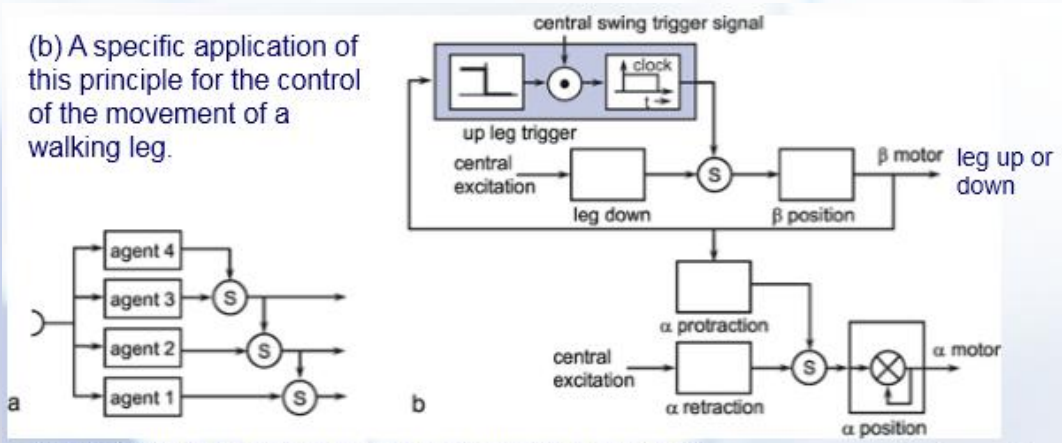
Single-Agent Architectures

- **Deliberative Agent Systems: Symbolic representation and manipulation**
  - **IRMA, GRATE**



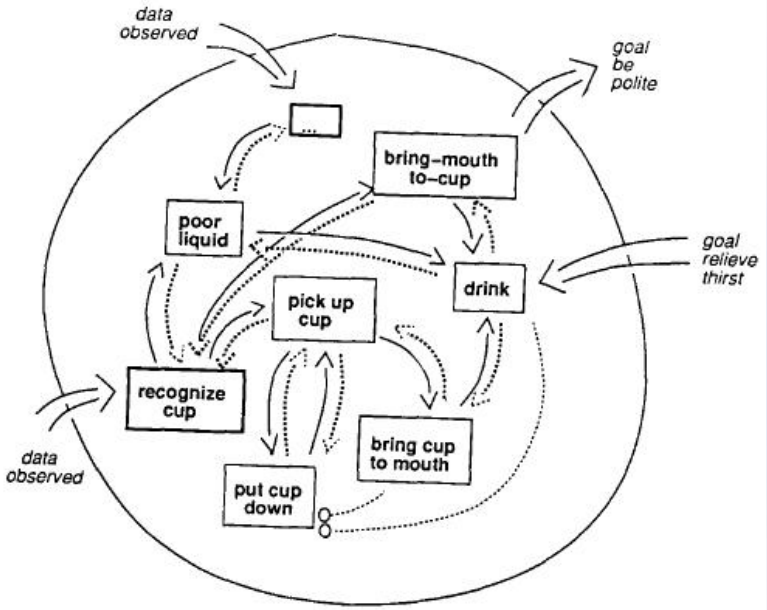
Single-Agent Architectures

- **Reactive: Stimulus -Response Agent Systems**
  - **Subsumption Architecture**
  - **Agent Network Architecture**



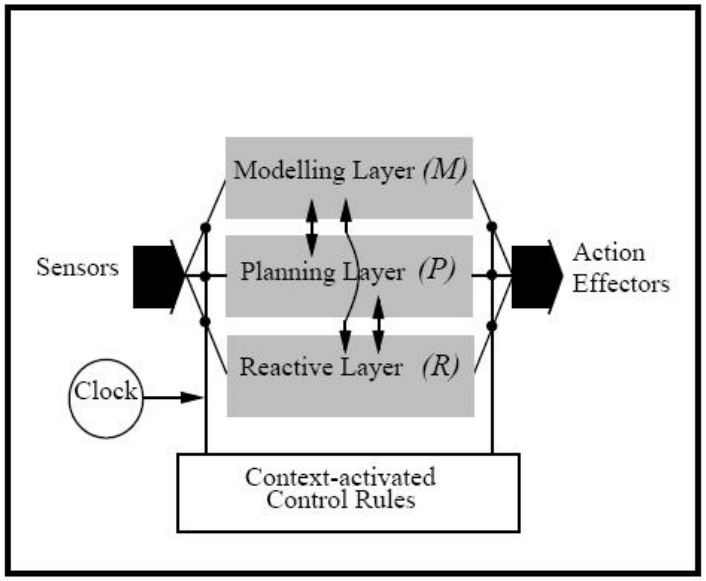
(a) Schematic illustration of Brook's subsumption architecture. The different agents are arranged in parallel, but determine the actual behavior in a hierarchical manner. A superior agent, if active, may suppress (S) the output of the inferior agent and take over the control of the behavior.

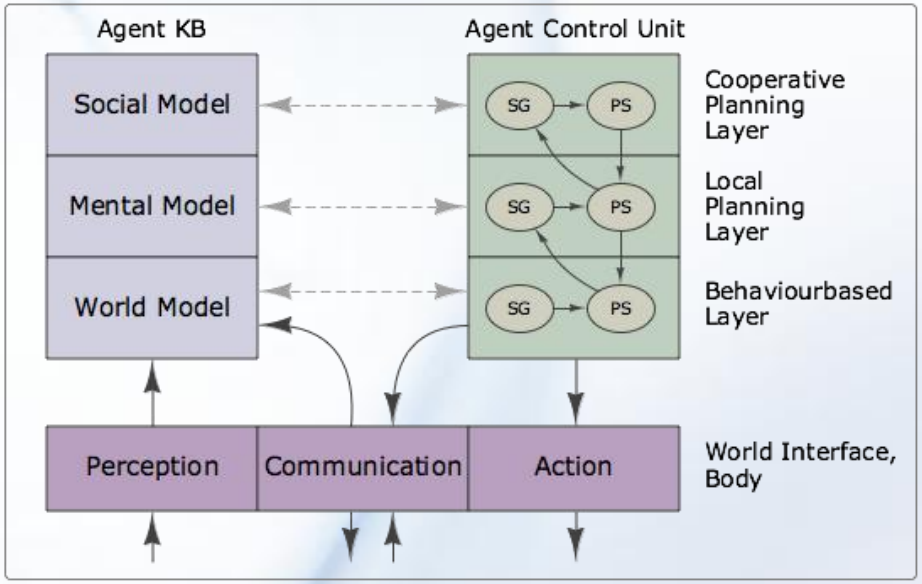
Holk Cruse: *Neural Networks as Cybernetic Systems (2nd edition), 2006*



Single-Agent Architectures

- **Hybrid: Act both deliberatively and reactively**
  - **PRS, TouringMachine, InteRRaP**





Part Ⅱ: Reinforcement Learning

Learning of Agents

- **Learning takes place as a result of interaction between an agent and the world, the idea behind learning is that**
  - Percepts received by an agent should be used not only for acting, but also for improving the agent's ability to behave optimally in the future to achieve the goal.

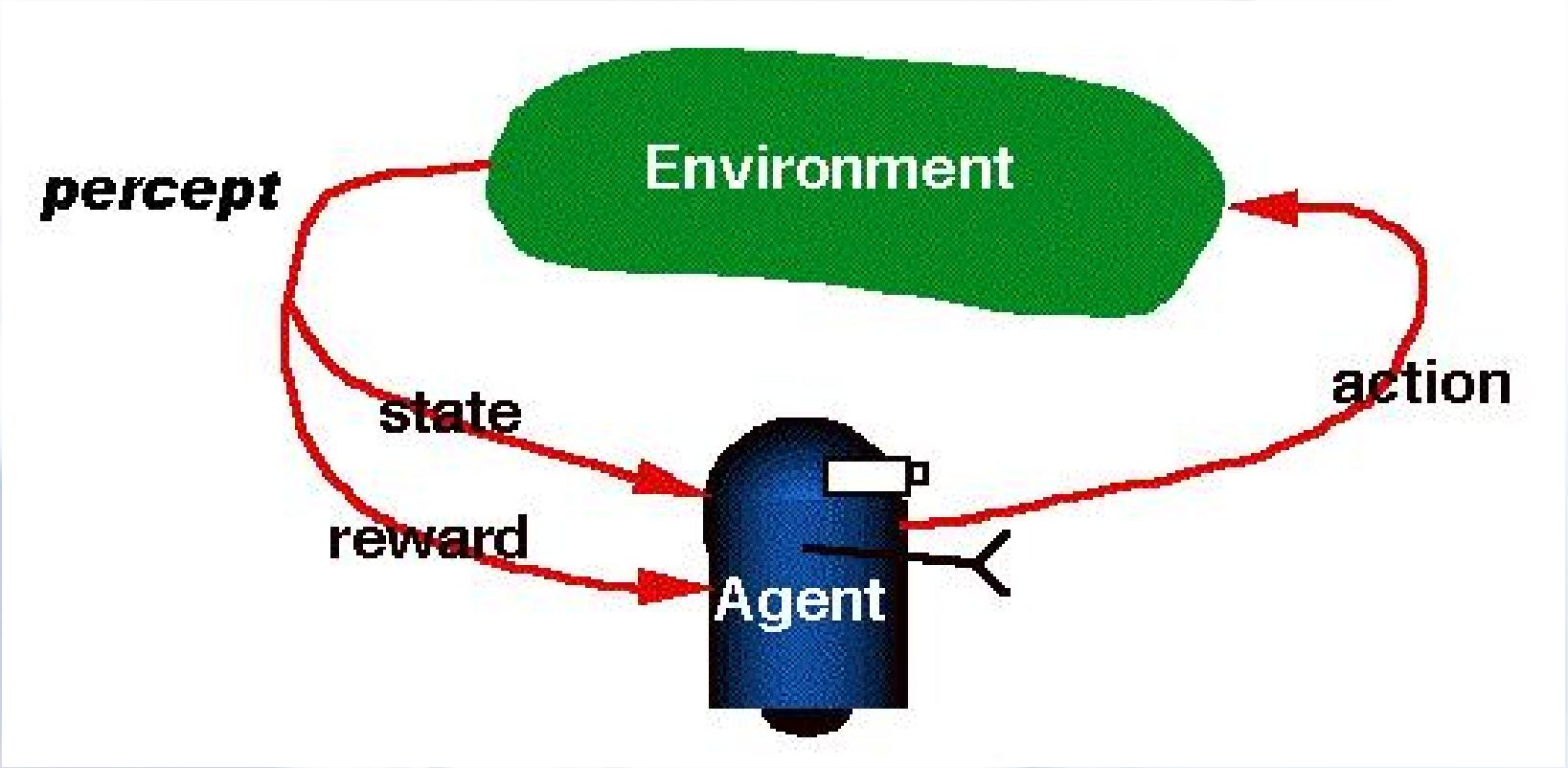
Learning Types

- **Supervised Learning**
  - (Input, output) pairs of the function to be learned can be perceived or are given.
- **Unsupervised Learning**
  - No information at all about given output
- **Reinforcement Learning** √
  - in the case of the agent acts on its environment, it receives some evaluation of its action (reinforcement), but is not told of which action is the correct one to achieve its goal

2.1 Introduction to RL Problem

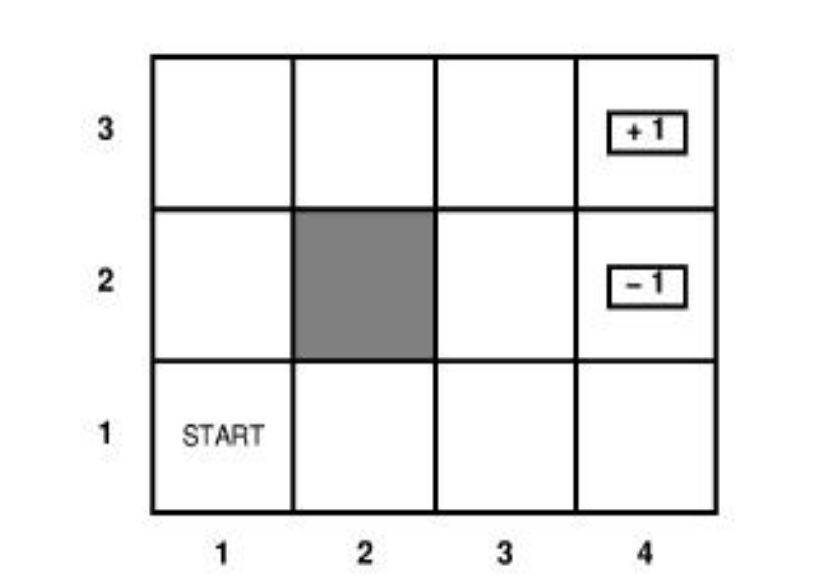
- **RL brings a way of programming agents by reward and punishment without specifying *how* the task is to be achieved. (Kaelbling,1996)**
  - In which we examine how an agent can learn from success and failure, reward and punishment.
  - Based on trial-error interactions

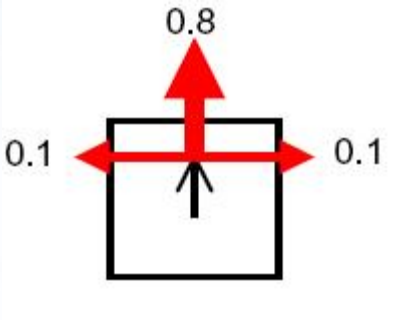
RL Model



Picture：R. Sutton: Reinforcement Learning: A Tutorial

Classical Example-Robot in a room

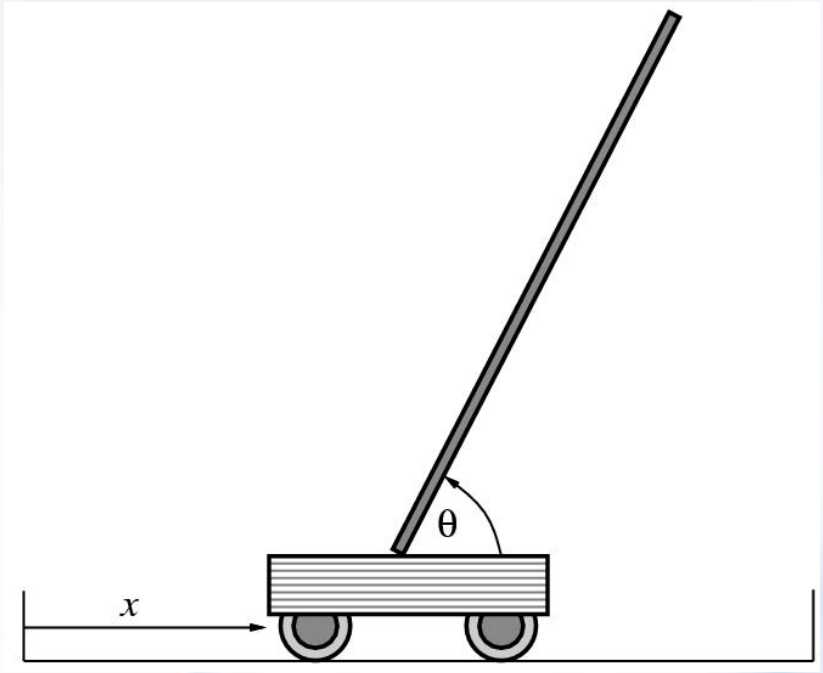




- **UP Action: 80% move UP, 10% move LEFT, 10% move RIGHT**
- **reward +1 at [4,3], -1 at [4,2], reward 0 for each step**

Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 2ed edition,2006

Classical Example – Pole Balancing

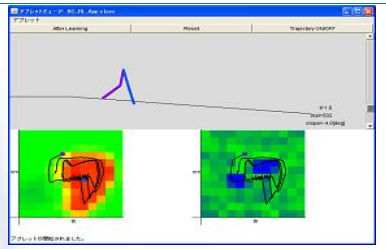


Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 2ed edition,2006

- **set up the problem of balancing a long pole upright on the top of a moving cart.**

Other Classical Examples

- **Walking robot (applet)**
- **Pendulum (applet)**
- **TD-Gammon [Gerry Tesauro]**
- **Helicopter [Andrew Ng]**

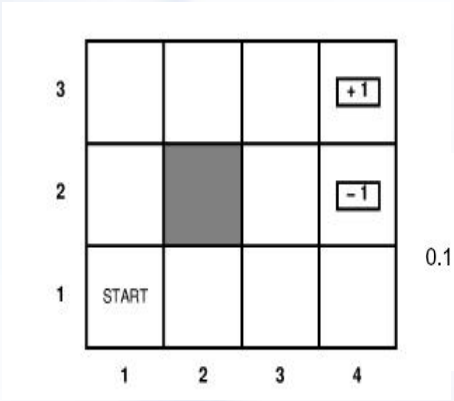




- **Task restated**
  - no teacher who would say “good” or “bad” (reward could be delayed)
  - explore the environment and learn from the experience (not just blind search, try to be smart about it).

2.2 Define an RL Problem: Markov Decision Process (MDP)





$$s_0 \xrightarrow{a_0 : r_0} s_1 \xrightarrow{a_1 : r_1} s_2 \xrightarrow{a_2 : r_2} \cdots$$

- **Transition model T(s,a,s'), how action influence states, e.g., T([1,1], up, [1,2]) = 0.8.**
- **Reward R(s), immediate value of state-action transition, e.g., r([4,3]) = +1.**
- **Policy π(s) or π(s,a) , maps states to actions**

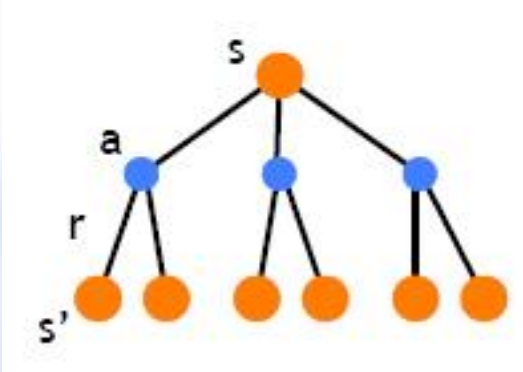
GOAL: maximize cumulative reward in the long run (optimal policy)

Computing return from rewards

- **additive rewards**
  - Equation: $V^{\pi}(s_t) = \sum_{i=0}^{h} r_{t+i}$
  - infinite value for continuing tasks
- **average rewards**
  - Equation: $V^{\pi}(s_t) = \lim_{h \to \infty} \frac{1}{h} \sum_{i=0}^{h} r_{t+i}$
- **discounted rewards**
  - Equation: $V^{\pi}(s_t) = \sum_{i=0}^{\infty} \gamma^i r_{t+i}, 0 < \gamma \leq 1$
  - value bounded if rewards bounded

Value functions

- **state value function: $V^{\pi}(s)$**
  - expected return when starting in *s* and following π
- **state-action value function: $Q^{\pi}(s,a)$**
  - expected return when starting in *s*, performing *a,* and following π
    - useful for finding the optimal policy
      - can estimate from experience
      - pick the best action using Qπ(s,a)
- **Bellman equation**



$$V^*(s) = \max_a \left(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} T(s,a,s') V^*(s') \right), \forall s \in \mathcal{S}$$

Optimal value functions

- **there's a set of *optimal* policies**
  - V^{π} defines partial ordering on policies
  - they share the same optimal value function

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

- **Bellman optimality equation**
  - system of n non-linear equations
  - solve for V*(s)
  - extract the optimal policy

$$\pi^*(s) = \arg\max_a \left(R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V^*(s') \right)$$

- **having Q*(s,a) makes it even simpler**

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

Calculating the Optimal Policy

initialize $V(s)$ arbitrarily
loop until policy good enough
    loop for $s \in \mathcal{S}$
        loop for $a \in \mathcal{A}$
            $Q(s,a) := R(s,a) + \gamma \sum_{s' \in \mathcal{S}} T(s,a,s') V(s')$
        $V(s) := \max_a Q(s,a)$
    end loop
end loop

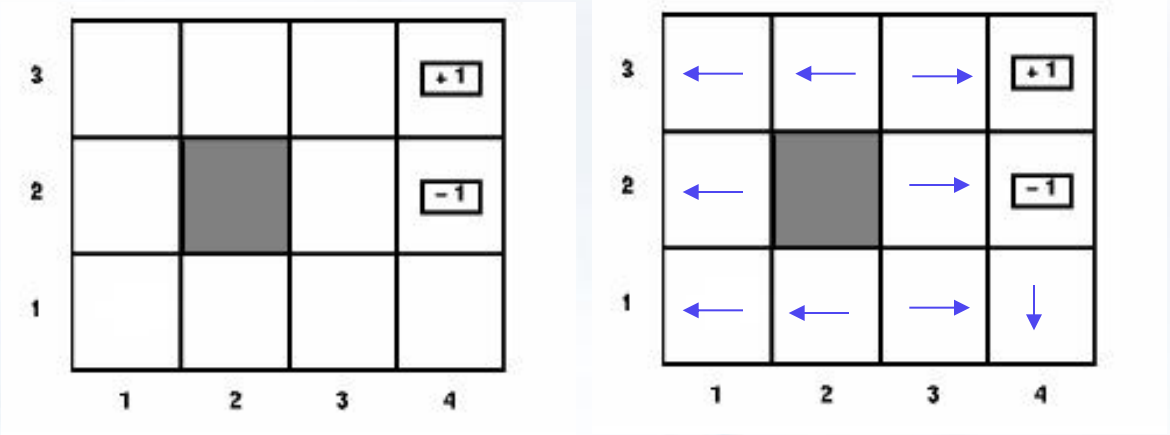
Value Iteration Algorithm

Policy Iteration Algorithm

choose an arbitrary policy π'
loop
    $\pi := \pi'$
    compute the value function of policy π:
        solve the linear equations
            $V_\pi(s) = R(s,\pi(s)) + \gamma \sum_{s' \in \mathcal{S}} T(s,\pi(s),s') V_\pi(s')$
    improve the policy at each state:
        $\pi'(s) := \arg\max_a \left(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} T(s,a,s') V_\pi(s')\right)$
until $\pi = \pi'$

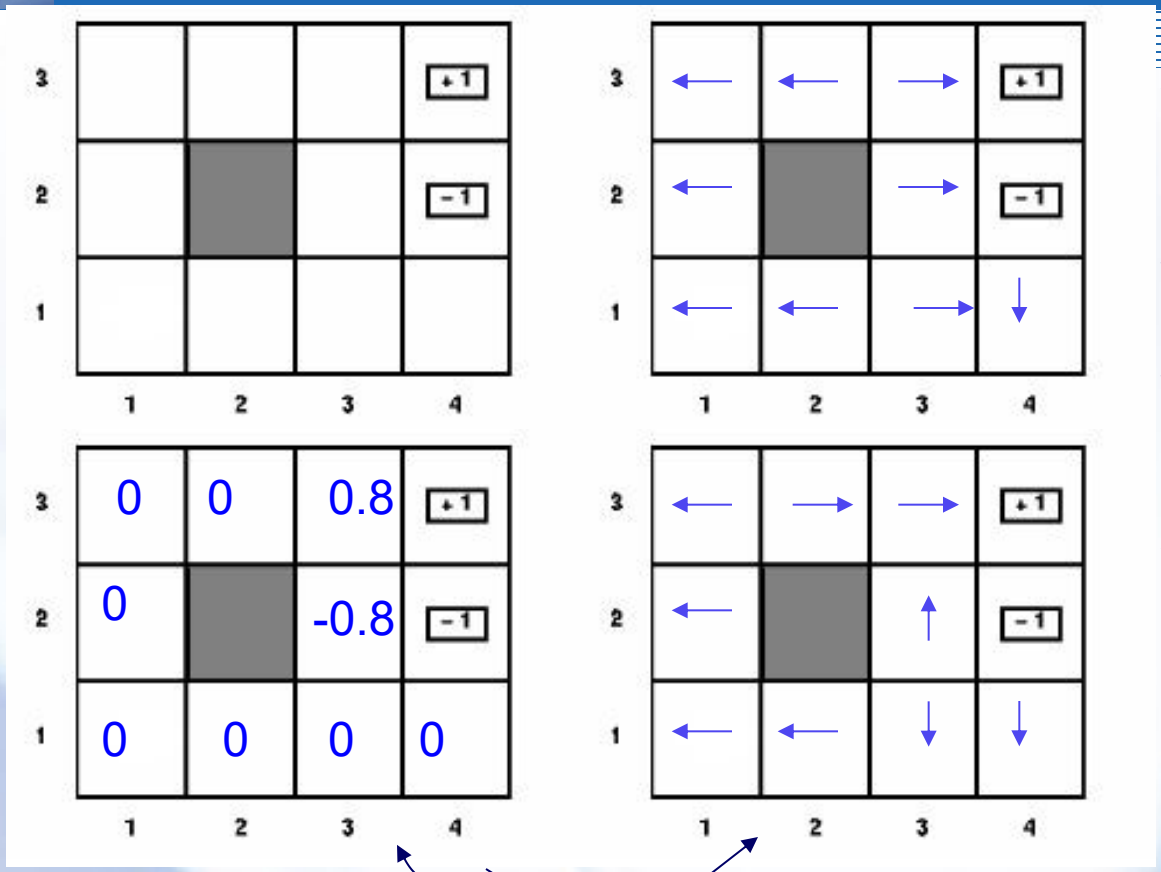
Tayfun Gürel:
Reinforcement Learning

POLICY ITERATION ON GRID WORLD



Tayfun Gürel:
Reinforcement Learning

POLICY ITERATION ON GRID WORLD



$$\pi^*(s) = \arg\max_a \left(R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V^*(s') \right)$$

$V^{\pi}(1,1) = 0.1 * V^{\pi}(1,2) + 0.1 * V^{\pi}(1,1)$

⋮

$V^{\pi}(3,2) = 0.8 * V^{\pi}(4,2) + 0.1 * V^{\pi}(3,3) + 0.1 * V^{\pi}(3,1)$

$V^{\pi}(3,3) = 0.8 * V^{\pi}(4,3) + 0.1 * V^{\pi}(3,2) + 0.1 * V^{\pi}(3,3)$

⋮

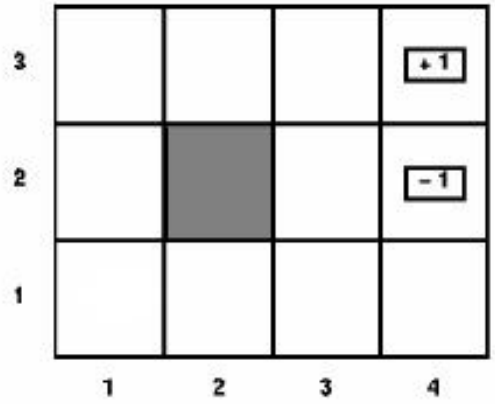
$V^{\pi}(4,4) = 1$

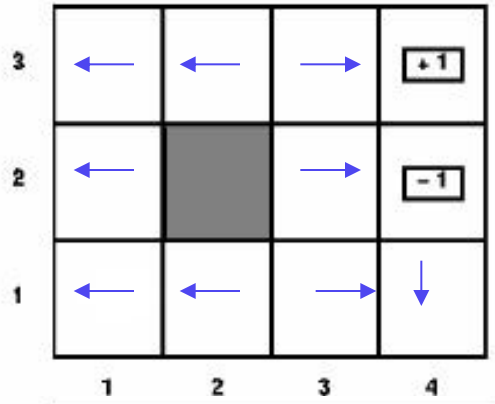
Solution:

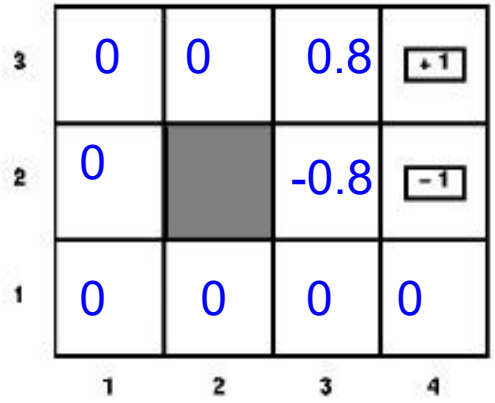
1. *n* linear equations with *n* unkowns
2. Iterative Updation

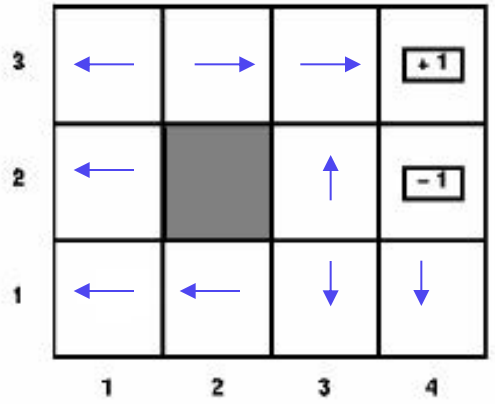
Picture: Tayfun Gürel: Reinforcement Learning

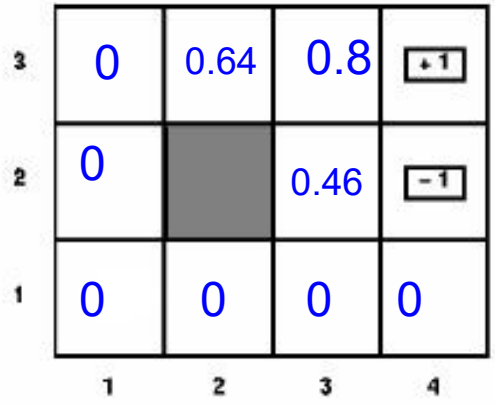
POLICY ITERATION ON GRID WORLD

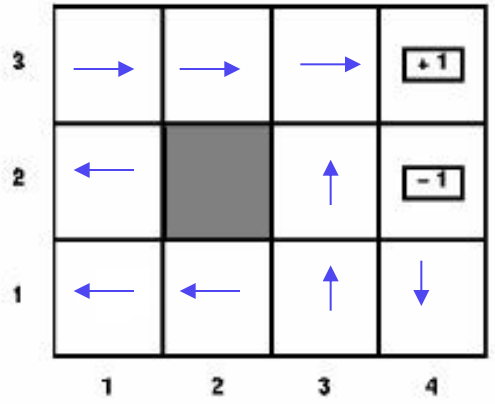






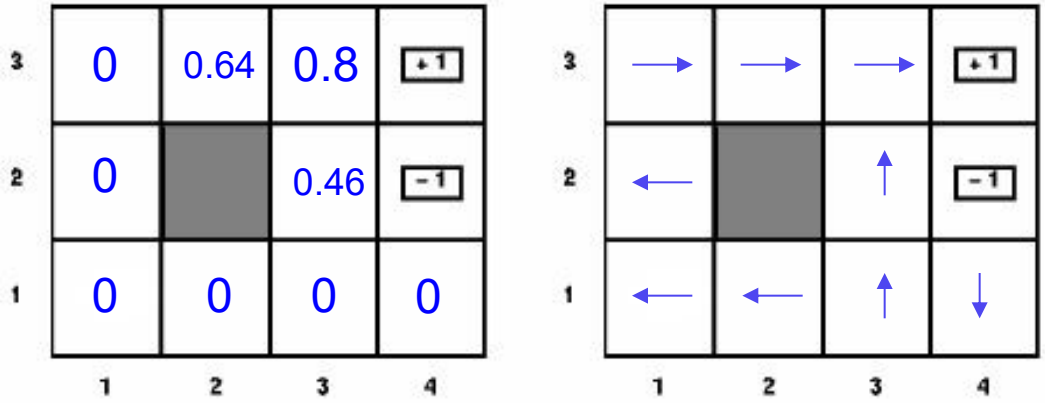






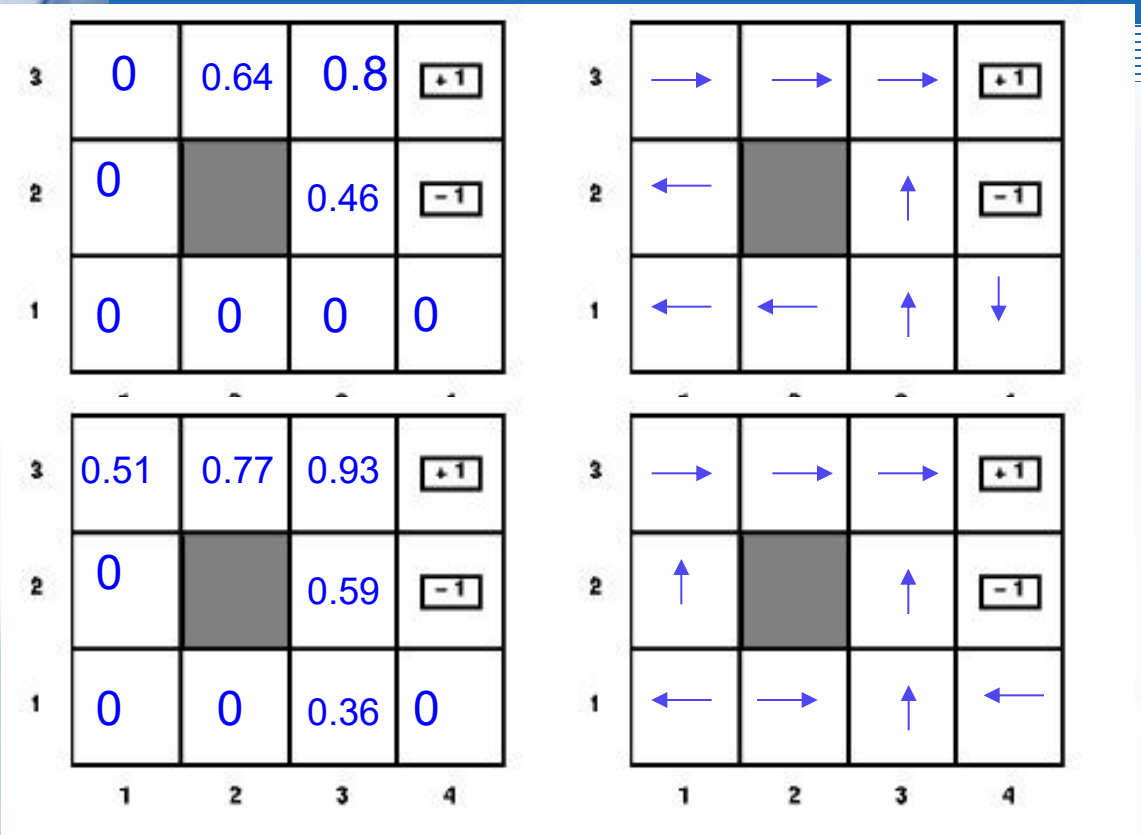
Tayfun Gürel:
Reinforcement Learning

POLICY ITERATION ON GRID WORLD



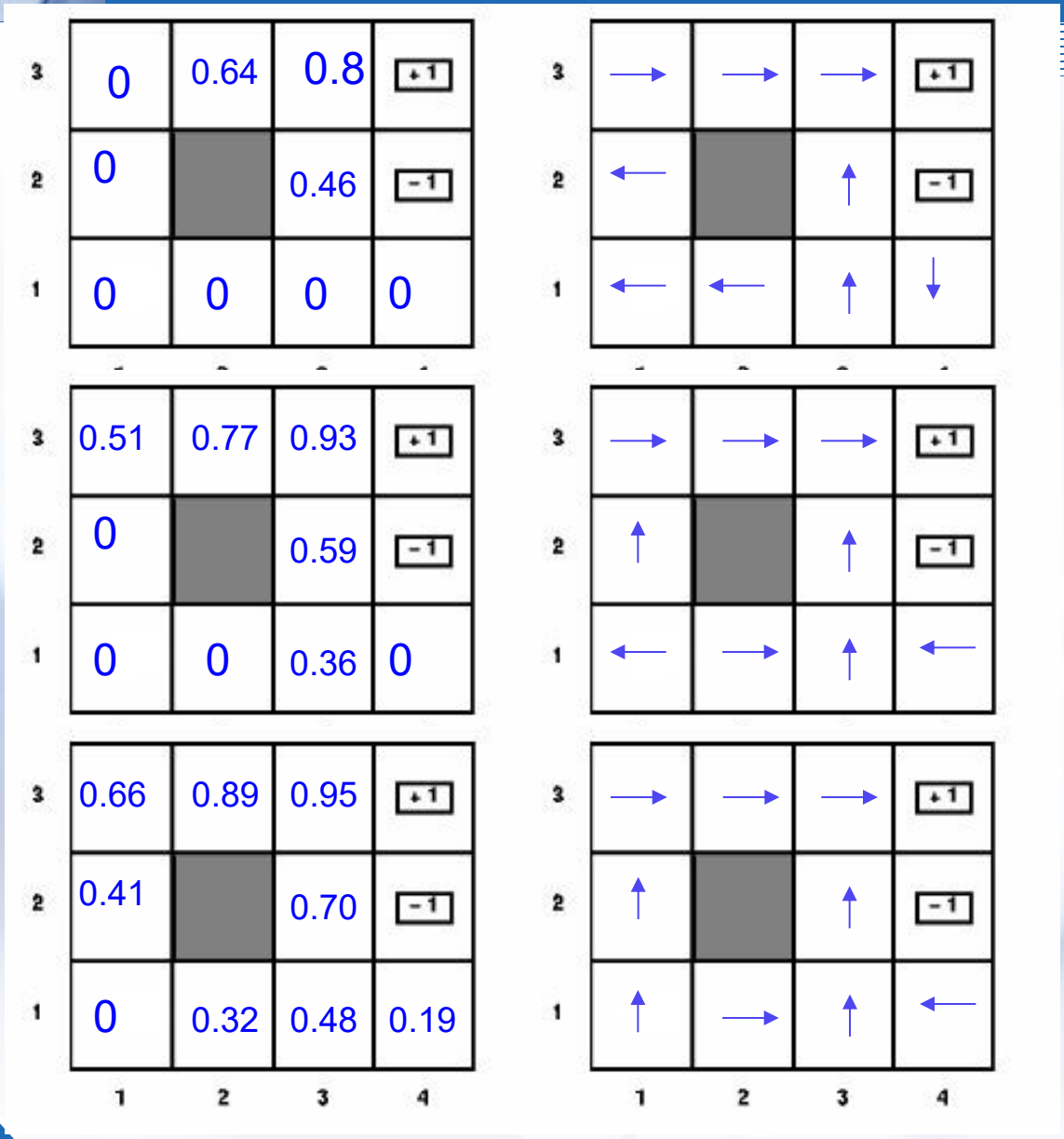
Tayfun Gürel:
Reinforcement Learning

POLICY ITERATION ON GRID WORLD



Tayfun Gürel:
Reinforcement Learning

POLICY ITERATION ON GRID WORLD



Tayfun Gürel:
Reinforcement Learning

2.3 Q-Learning

- **But, we don't know anything about the environment model—the transition function *T(s,a,s')* , Here comes two approaches**
  - ***Model based approach RL*:**

    learn the model, and use it to derive the optimal policy. e.g, Sarsa, Dyna-Q.
  - ***Model free approach RL*:**

    derive the optimal policy without learning the model. e.g, Temporal Difference (TD), Q-learing.

Model free Approch: Q-Learning

- Try to learn V^{π^*} (abbreviated V^*)
- Perform look ahead search to choose best action from any state s

$$\pi^*(s_t) = \arg\max_a \left[f_R(s_t, a) + \gamma V^*(f_S(s_t, a)) \right]$$

- Works well if agent knows
  - $f_S : state \times action \rightarrow state$
  - $f_R : state \times action \rightarrow \mathrm{R}$
- When agent doesn't know f_S and f_R, cannot choose actions this way

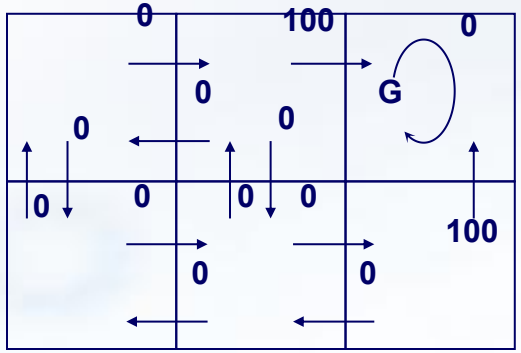
Q-values

- Define new function very similar to ***V****

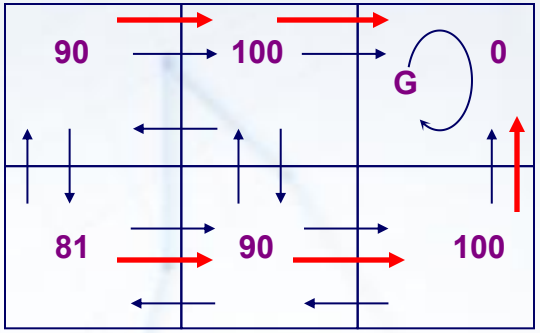
$$Q(s_t, a) = f_R(s_t, a) + \gamma V^*(f_S(s_t, a))$$

- If agent learns ***Q***, it can choose optimal action even without knowing f_S and f_R

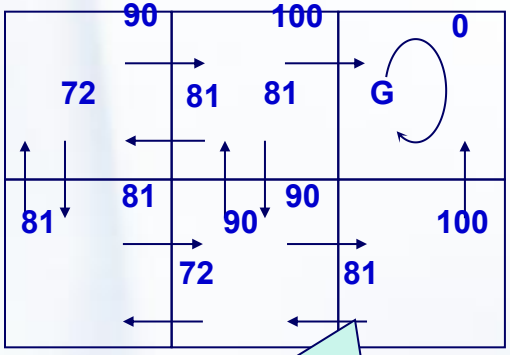
Q-values Computation



***r*(*state*, *action*) immediate reward values**



V(**state**) values*



***Q*(*st*... ...lues**

81=0+0.9*90

$$V^*(s_t) = \max_{\pi} V^{\pi}(s_t), \forall s_t \in \boldsymbol{S}$$

$$V^{\pi}(s_t) = \sum_{i=0}^{\infty} \gamma^i r_{t+i}$$

$$Q(s_t, a) = f_R(s_t, a) + \gamma V^*(f_S(s_t, a))$$

- Using the *discounted cumulative reward*, the discount factor = 0.9

Learning the Q-value

- **Note: *Q* and *V** closely related**

$$V^*(s) \equiv \arg\max_{a'} Q(s, a') \qquad Q(s_t, a) = f_R(s_t, a) + \gamma V^*(f_S(s_t, a))$$

- **Allows us to write *Q* recursively as**

$$Q(s_t, a) = f_R(s_t, a) + \gamma \max_{a'} Q(f_S(s_t, a), a')$$

Solve: How to learn?

- **Using Q-values**

$$\pi^*(s_t) = \arg\max_a Q(s_t, a)$$

Solve: How to choose the best action?

A kind of Temporal Difference (TD) learning

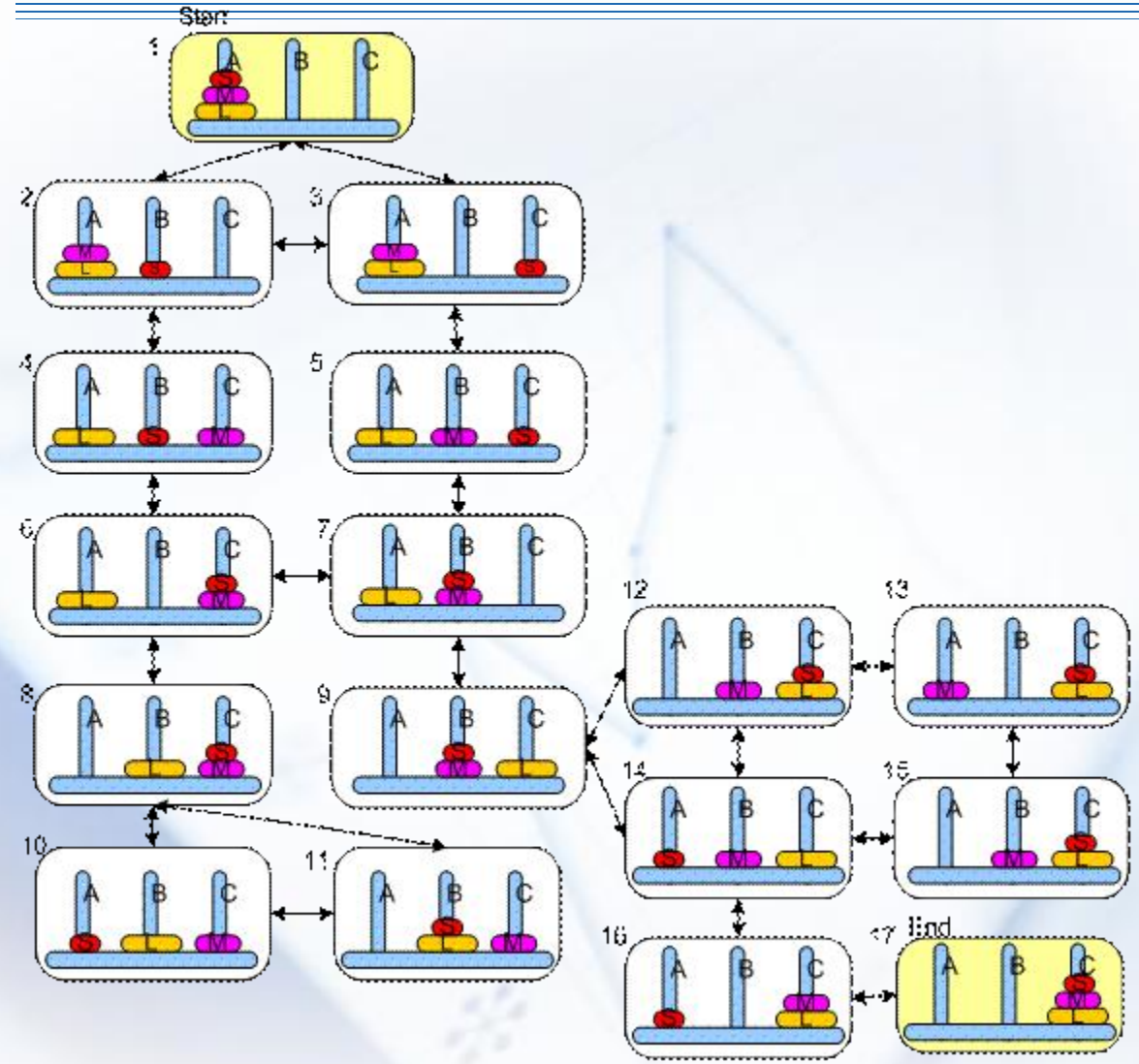
Q-Learning Steps

- **FOR each <*s*, *a*> DO**
  - *Initialize* table entry: $\hat{Q}(s, a) \leftarrow 0$
- **Observe current state *s***
- **WHILE (true) DO**
  - *Select* action *a* and execute it
  - *Receive* immediate reward *r*
  - *Observe* new state *s′*
  - *Update* table entry for $\hat{Q}(s, a)$ as follows

$$\hat{Q}(s, a) \leftarrow r(s, a) + \gamma \max_{a'} \hat{Q}(s', a')$$

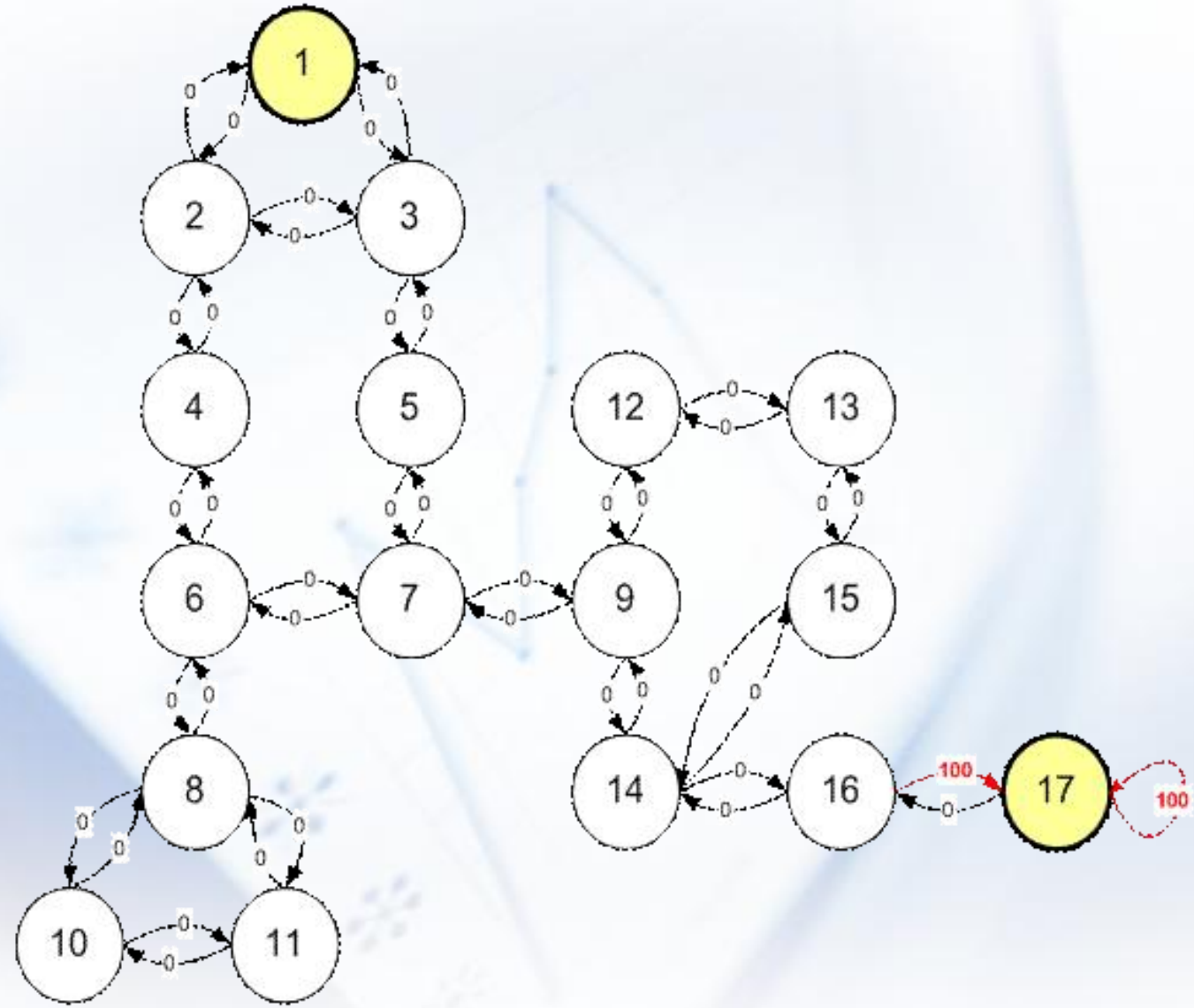
  - *Move*: record transition from *s* to *s′*

Q-Learning Example: Tower of Hanoi



http://people.revoledu.com/kardi/tutorial/ReinforcementLearning/Tower-of-Hanoi.htm

Q-Learning Example: State Diagram with Reward Values



Q-Learning Example: Computation of Q Values

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
13	State	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
14	1	-	0	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-
15	2	0	-	0	0	-	-	-	-	-	-	-	-	-	-	-	-	-
16	3	0	0	-	-	0	-	-	-	-	-	-	-	-	-	-	-	-
17	4	-	0	-	-	-	0	-	-	-	-	-	-	-	-	-	-	-
18	5	-	-	0	-	-	-	0	-	-	-	-	-	-	-	-	-	-
19	6	-	-	-	0	-	-	0	0	-	-	-	-	-	-	-	-	-
20	7	-	-	-	-	0	0	-	-	0	-	-	-	-	-	-	-	-
21	8	-	-	-	-	-	0	-	-	-	0	0	-	-	-	-	-	-
22	9	-	-	-	-	-	-	0	-	-	-	-	0	-	0	-	-	-
23	10	-	-	-	-	-	-	-	0	-	-	0	-	-	-	-	-	-
24	11	-	-	-	-	-	-	-	0	-	0	-	-	-	-	-	-	-
25	12	-	-	-	-	-	-	-	-	0	-	-	-	0	-	-	-	-
26	13	-	-	-	-	-	-	-	-	-	-	-	0	-	-	0	-	-
27	14	-	-	-	-	-	-	-	-	0	-	-	-	-	-	0	0	-
28	15	-	-	-	-	-	-	-	-	-	-	-	-	0	0	-	-	-
29	16	-	-	-	-	-	-	-	-	-	-	-	-	-	0	-	-	100
30	17	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	100

R Matrix

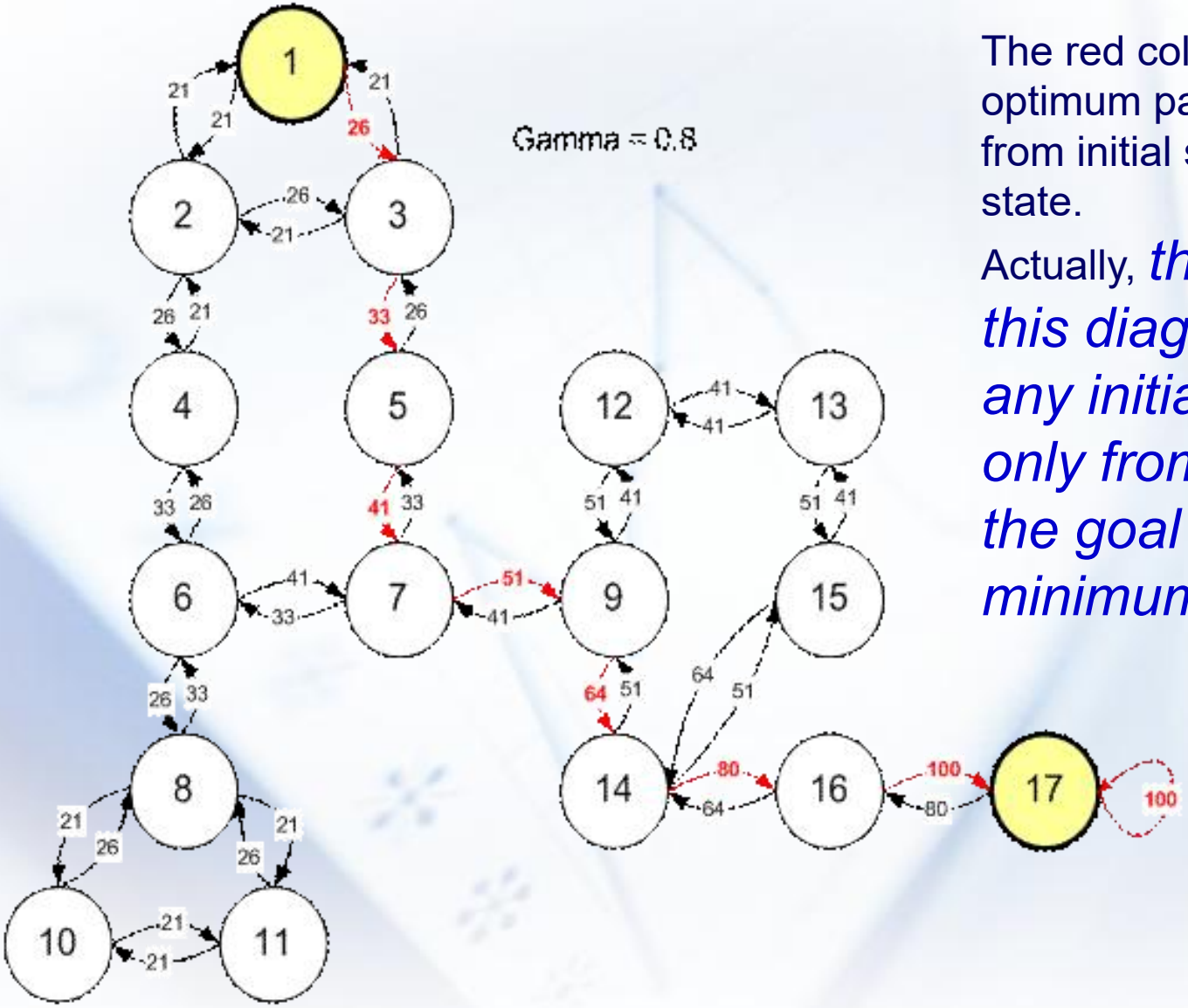
Initial Q

Q Matrix

Final Q

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
72	Norm Qn	Action																		
73	State	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	max	Max State
74	1	-	21%	26%	-	-	-	-	-	-	-	-	-	-	-	-	-	-	26%	3
75	2	21%	-	26%	26%	-	-	-	-	-	-	-	-	-	-	-	-	-	26%	3
76	3	21%	21%	-	-	33%	-	-	-	-	-	-	-	-	-	-	-	-	33%	5
77	4	-	21%	-	-	-	33%	-	-	-	-	-	-	-	-	-	-	-	33%	6
78	5	-	-	26%	-	-	-	41%	-	-	-	-	-	-	-	-	-	-	41%	7
79	6	-	-	-	26%	-	-	41%	26%	-	-	-	-	-	-	-	-	-	41%	7
80	7	-	-	-	-	33%	33%	-	-	51%	-	-	-	-	-	-	-	-	51%	9
81	8	-	-	-	-	-	33%	-	-	-	21%	21%	-	-	-	-	-	-	33%	6
82	9	-	-	-	-	-	-	41%	-	-	-	-	41%	-	64%	-	-	-	64%	14
83	10	-	-	-	-	-	-	-	26%	-	-	21%	-	-	-	-	-	-	26%	8
84	11	-	-	-	-	-	-	-	26%	-	21%	-	-	-	-	-	-	-	26%	8
85	12	-	-	-	-	-	-	-	-	51%	-	-	-	41%	-	-	-	-	51%	9
86	13	-	-	-	-	-	-	-	-	-	-	-	41%	-	-	51%	-	-	51%	15
87	14	-	-	-	-	-	-	-	-	51%	-	-	-	-	-	51%	80%	-	80%	16
88	15	-	-	-	-	-	-	-	-	-	-	-	-	41%	64%	-	-	-	64%	14
89	16	-	-	-	-	-	-	-	-	-	-	-	-	-	64%	-	-	100%	100%	17
90	17	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	80%	100%	100%	17

Q-Learning Example: the solution in the state diagram



The red color of arrow is the optimum path (minimum path) from initial state to the goal state.

Actually, *the Q values in this diagram will lead to any initial state (not only from state 1) to the goal state using minimum path*

Q-Learning Demo: PathLearner



Demo from: http://www2.hawaii.edu/~chenx/ics699rl/grid/

2.4 TD(lambda) Learning

- Recall the supervised learning based on gradients: for a sequence of input pairs, we have,

$$\omega = \omega + \sum_{i=1}^{m} \Delta\omega_t \qquad \Delta\omega_t = \alpha\left(z - Y_t\right)\nabla_{\omega} Y_t$$

- However, we can only observe the result until the end of the sequence for RL problem

TD(lambda) Learning

- So we represent the error as a sum of changes in predictions,

$$z - Y_t = \sum_{k=t}^{m} \left(Y_{k+1} - Y_k \right), \text{ where } Y_{m+1} = z$$

- Thus,

TD(lambda) Learning

$$\omega = \omega + \sum_{t=1}^{m} \Delta\omega_t \Rightarrow \omega = \omega + \sum_{t=1}^{m} \alpha\left(z - Y_t\right)\nabla_\omega Y_t$$

- Accordingly, we can conduct learning incrementally as SL:

$$\Delta\omega_t = \alpha\left(Y_{t+1} - Y_t\right)\sum_{k=1}^{t} \nabla_\omega P_k$$

$$= \omega + \sum_{t=1}^{m} \alpha \sum_{k=t}^{m} \left(Y_{k+1} - Y_k\right)\nabla_\omega Y_t$$

$$= \omega + \sum_{k=1}^{m} \alpha \sum_{t=1}^{k} \left(Y_{k+1} - Y_k\right)\nabla_\omega Y_t$$

$$= \omega + \sum_{t=1}^{m} \alpha\left(Y_{t+1} - Y_t\right)\sum_{k=1}^{t} \nabla_\omega Y_k$$

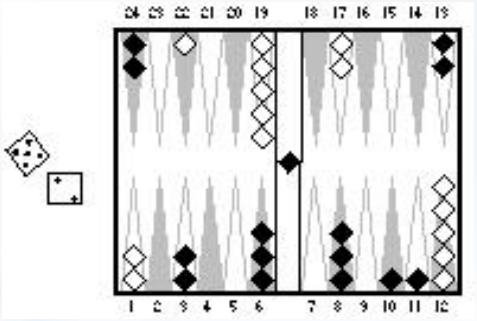
TD(lambda) Learning

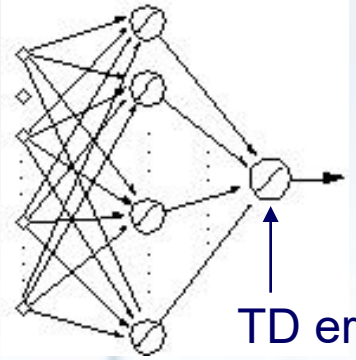
- By considering the discount along with the time and introduces the weight, i.e., lambda, we get

$$\Delta\omega_t = \alpha\left(Y_{t+1} - Y_t\right)\sum_{k=1}^{t}\lambda^{t-k}\nabla_{\omega}P_k$$

- Now we have TD(lambda) learning algorithm

Practical Applications: TD-Gammon





Action selection
by 2–3 ply search

Start with a random network

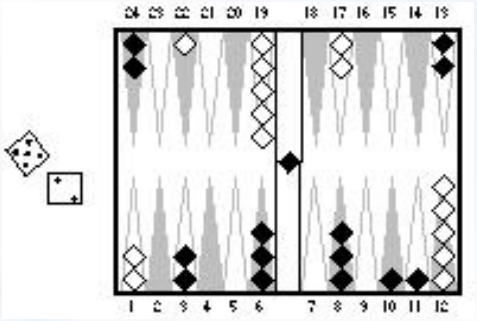
Play millions of games against self

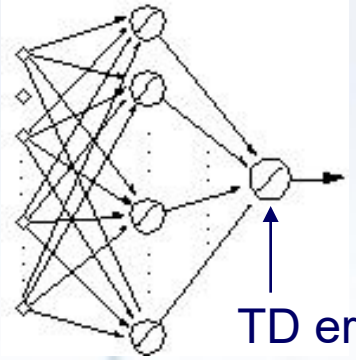
Learn a value function from this simulated experience

This produces arguably the best player in the world

G. Tesauro. Temporal Difference Learning and TD-Gammon By Gerald Tesauro. Communications of the ACM, 1995, 38(3)

Practical Applications: TD-Gammon





Action selection
by 2–3 ply search

Learning methodology: TD(lambda) algorithm

$$w_{t+1} - w_t = \alpha(Y_{t+1} - Y_t) \sum_{k=1}^{t} \lambda^{t-k} \nabla_w Y_k$$

Network output

Graident

Practical Applications: TD-Gammon

Learning Result:

Program	Training Games	Opponents	Results
TDG 1.0	300,000	Robertie, Davis, Magriel	-13 pts/51 games (-0.25 ppg)
TDG 2.0	800,000	Goulding, Woolsey, Snellings, Russell, Sylvester	-7 pts/38 games (-0.18 ppg)
TDG 2.1	1,500,000	Robertie	-1 pt/40 games (-0.02 ppg)

Table 1. Results of testing TD-Gammon in play against world-class human opponents. Version 1.0 used 1-play search for move selection; versions 2.0 and 2.1 used 2-ply search. Version 2.0 had 40 hidden units; versions 1.0 and 2.1 had 80 hidden units.

2.5 REINFORCE

- REINFORCE is a RL algorithm based on the gradients of policy likelihood functions.

"*RE*ward *I*ncrement = *N*onnegative *F*actor × *O*ffset *R*einforcement × *C*haracteristic *E*ligibility

$$\Delta\omega_{ij} = \alpha_{ij}\left(r - b_{ij}\right)\sum_{t=1}^{k} e_{ij}\left(t\right)$$

reward

baseline, usually is 0

$$e_{ij}\left(t\right) = \frac{\partial \ln g_i}{\partial \omega_{ij}}$$

$$g_i(\xi, \mathbf{w}^i, \mathbf{x}^i) = Pr\{y_i = \xi \,|\, \mathbf{w}^i, \mathbf{x}^i\}$$

2.5 REINFORCE

Theorem 1. For any REINFORCE algorithm, the inner product of $E\{\Delta \mathbf{W} \mid \mathbf{W}\}$ and $\nabla_{\mathbf{W}} E\{r \mid \mathbf{W}\}$ is nonnegative. Furthermore, if $\alpha_{ij} > 0$ for all i and j, then this inner product is zero only when $\nabla_{\mathbf{W}} E\{r \mid \mathbf{W}\} = 0$. Also, if $\alpha_{ij} = \alpha$ is independent of i and j, then $E\{\Delta \mathbf{W} \mid \mathbf{W}\} = \alpha \nabla_{\mathbf{W}} E\{r \mid \mathbf{W}\}$.

This results relates $\nabla_{\mathbf{W}} E\{r \mid \mathbf{W}\}$, the gradient in weight space of the performance measure $E\{r \mid \mathbf{W}\}$, to $E\{\Delta \mathbf{W} \mid \mathbf{W}\}$, the average update vector in weight space, for any REINFORCE algorithm.[1] Specifically, it says that for any such algorithm the average update vector in weight space lies in a direction for which this performance measure is increasing. The last sentence of the theorem is equivalent to the claim that for each weight w_{ij} the quantity $(r - b_{ij})\, \partial \ln g_i / \partial w_{ij}$ represents an unbiased estimate of $\partial E\{r \mid \mathbf{W}\} / \partial w_{ij}$.

$$\Delta \omega_{ij} = \alpha_{ij} \left(r - b_{ij} \right) \sum_{t=1}^{k} e_{ij}(t)$$

Practical Applications: AlphaGo

- First Reinforcement Learning: Improve through self-plays, optimize the policy network



The RL policy network won more than 80% of games against the SL policy network

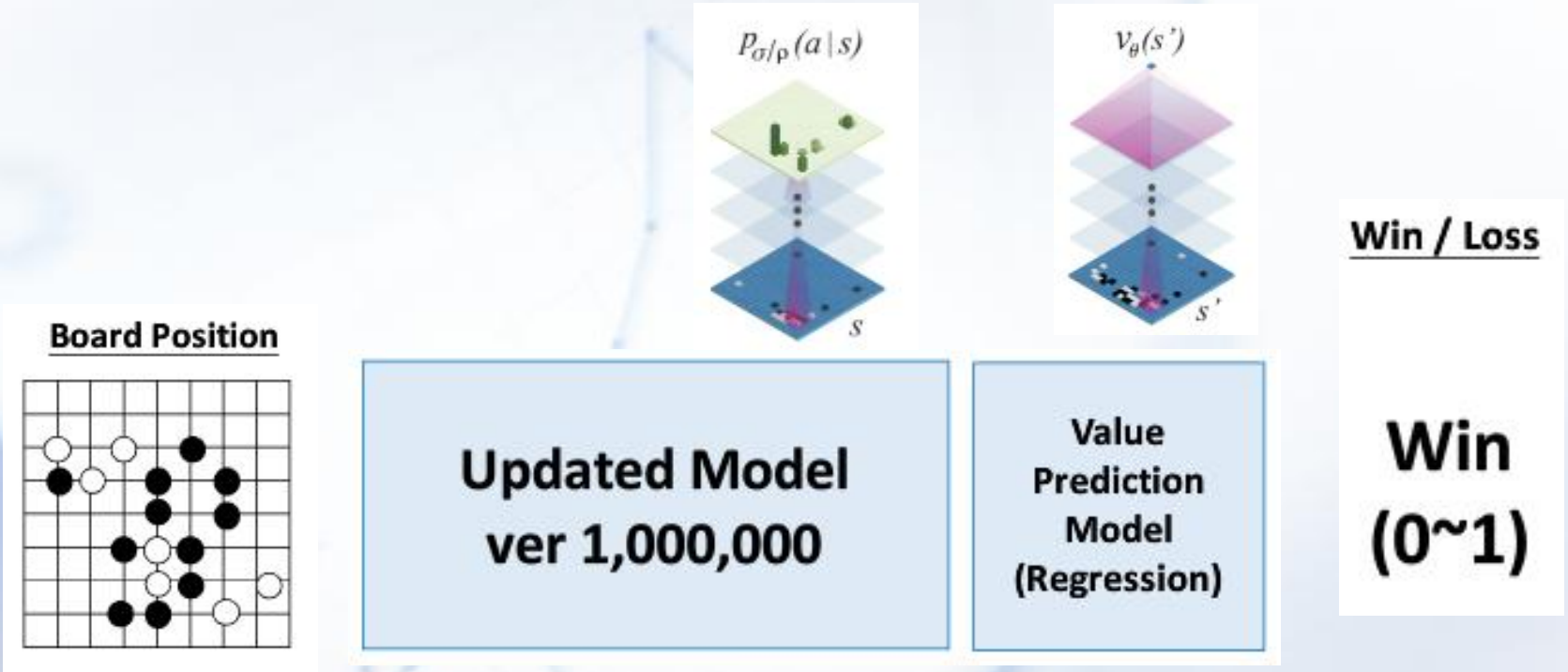
$$\rho = \rho + \eta \frac{\partial \log p_\rho(a_t|s_t)}{\partial \sigma} z_t$$

Objectiveness: MLE

Optimization: stochastic gradient ascent

Practical Applications: AlphaGo

- Second Reinforcement Learning: Board Evaluation, optimize the value network



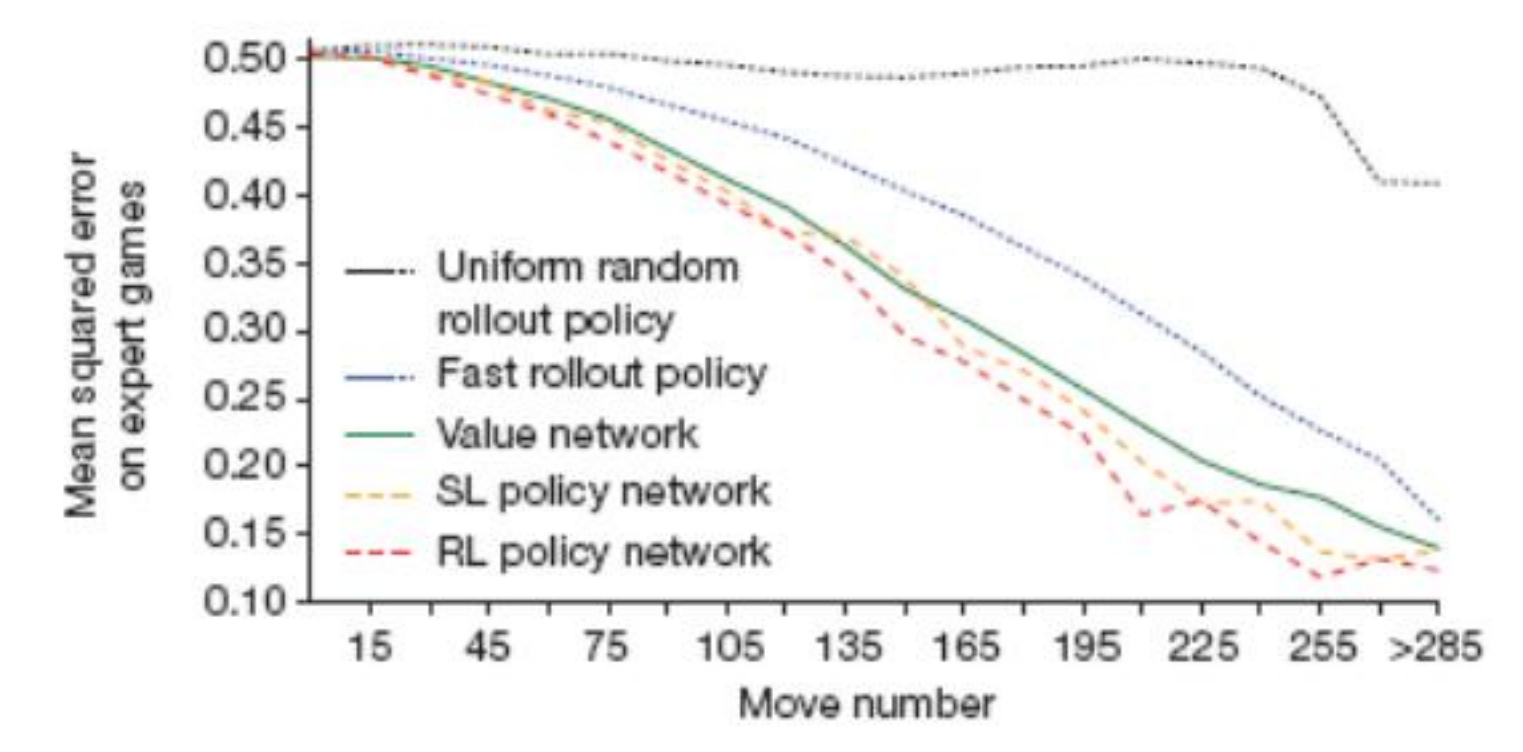
$$\theta = \theta - \eta \frac{\partial v_\theta(s)}{\partial \theta}\left(z - v_\theta(s)\right)$$

Objectiveness: MSE

Optimization: stochastic gradient descent

Practical Applications: AlphaGo

- Learning result of value network:



Part Ⅲ: Deep Reinforcement Learning

Deep Reinforcement Learning

❖ Value-based DRL

- Deep Q Network (DQN)
- Double Deep Q Network (DDQN)
- Dueling Deep Q Network (Dueling DQN)

❖ Policy-Based DRL

- Policy Gradient (PG)
- Asynchronous Reinforcement Learning (A3C)
- Deep Deterministic Policy Gradient (DDPG)

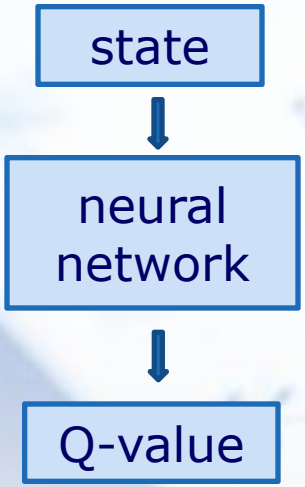
DRL = Using deep network to realize the reinforcement learning

3.1 Value-based DRL

- **Deep Q Network (DQN)**

DQN uses a neural network to calculate and output Q-values instead of traditional Q-table.

Deep Q-learning

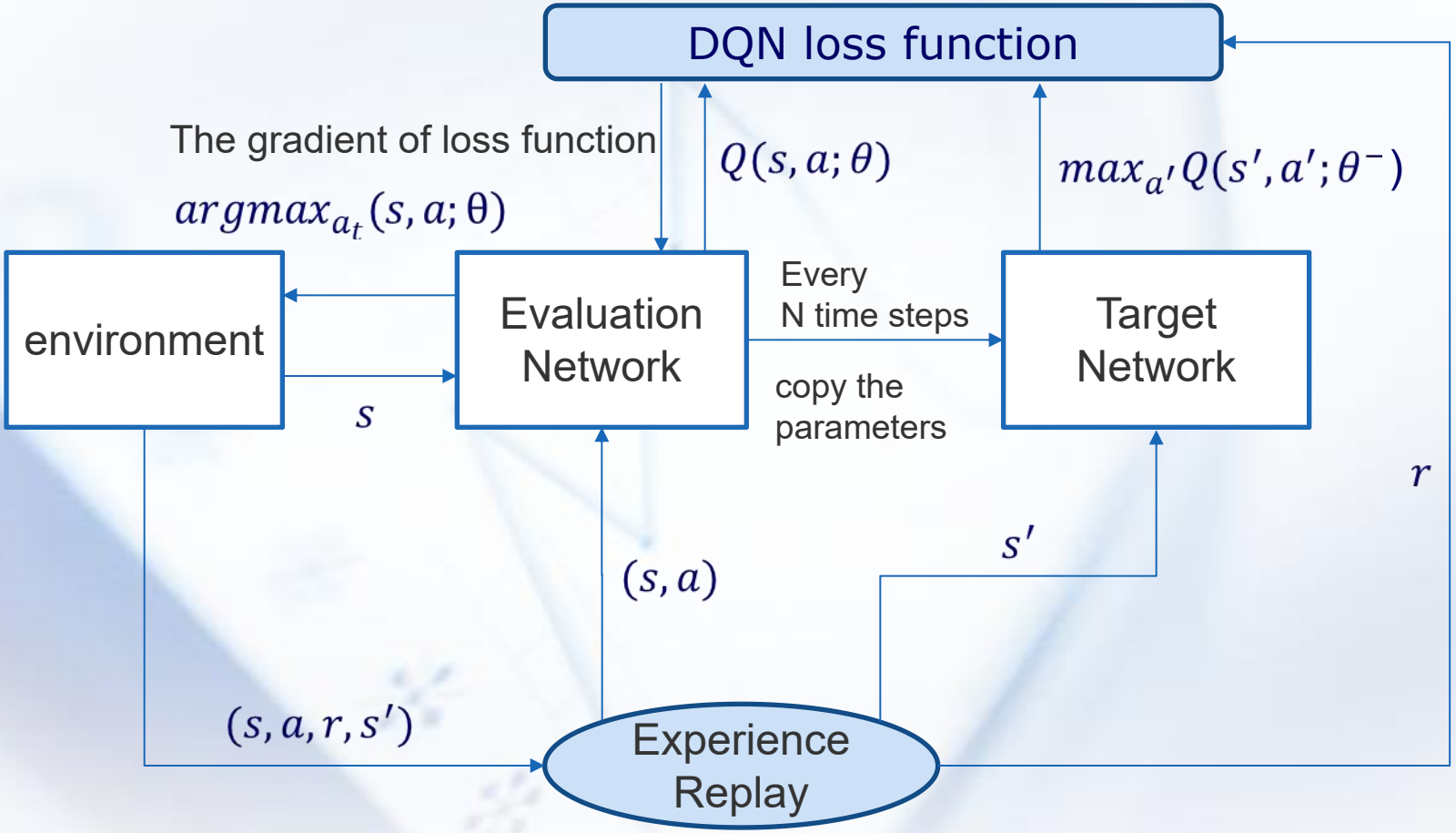


Q-learning

Q-table	action1	action2
state1	q(s1,a1)	q(s1,a2)
state2	q(s2,a1)	q(s2,a2)
state3	q(s3,a1)	q(s3,a2)

Value-based DRL

- **Deep Q Network (DQN)**



Value-based DRL

- **Deep Q Network (DQN)**

Algorithm 1 Deep Q-learning with experience replay

Initialize replay memory D to capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$
for episode 1, M **do** Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ε select a random action a_t
    otherwise select $a_t = \arg\max_a Q(\phi(s_t), a; \theta)$
    Execute action a_t in the emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store experience $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
    Sample random minibatch of experiences $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the weights θ
    Every C steps reset $\hat{Q} = Q$
  end for
end for

Value-based DRL

- **Double Deep Q Network (DDQN)**

$$Y_t^{\mathrm{Q}} \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \boldsymbol{\theta}_t)$$

$$Y_t^{\mathrm{DQN}} \equiv R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \boldsymbol{\theta}_t^-) .$$

$$Y_t^{\mathrm{Q}} = R_{t+1} + \gamma Q(S_{t+1}, \operatorname*{argmax}_a Q(S_{t+1}, a; \boldsymbol{\theta}_t); \boldsymbol{\theta}_t)$$

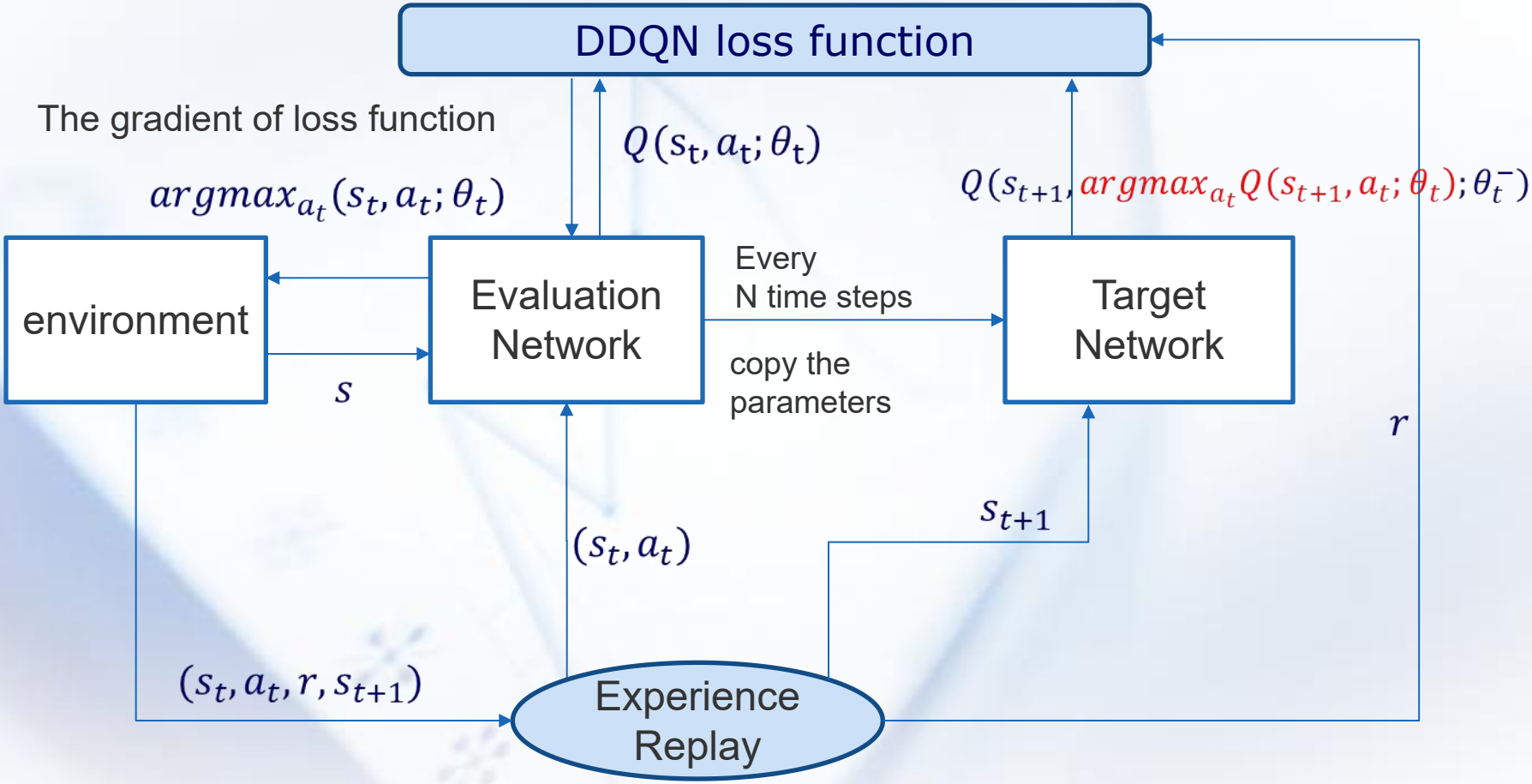
$$Y_t^{\mathrm{DoubleQ}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \operatorname*{argmax}_a Q(S_{t+1}, a; \boldsymbol{\theta}_t); \boldsymbol{\theta}_t')$$

The max operator in standard Q-learning and DQN, in (2) and (3), uses the same values both to select and to evaluate an action. This makes it more likely to select overestimated values, resulting in overoptimistic value estimates. To prevent this, we can decouple the selection from the evaluation.

In Double Q-learning (van Hasselt 2010), two value functions are learned by assigning experiences randomly to update one of the two value functions, resulting in two sets of weights, $\boldsymbol{\theta}$ and $\boldsymbol{\theta}'$. For each update, one set of weights is used to determine the greedy policy and the other to determine its value. For a clear comparison, we can untangle the selection and evaluation in Q-learning and rewrite its target (2) as

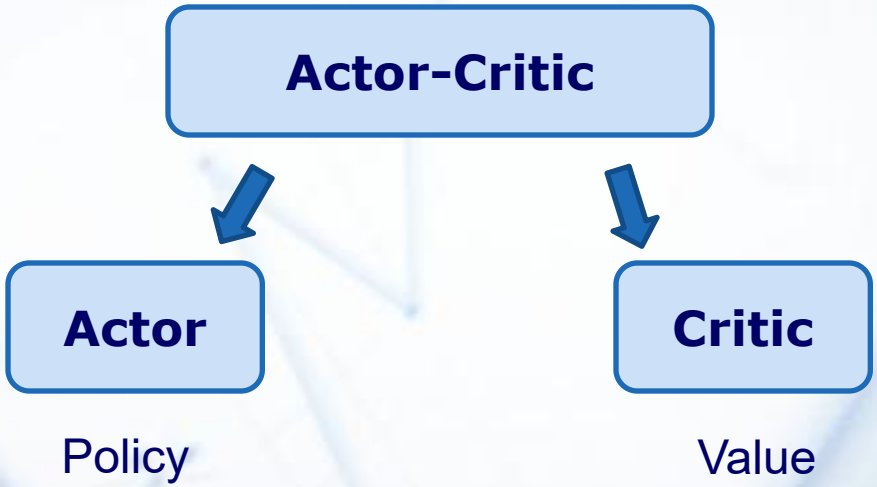
Value-based DRL

- **Double Deep Q Network (DDQN)**



3.2 Policy-based DRL

- **Actor-Critic (AC)**
- **Advantage Actor-Critic (A2C)**
- **Asynchronous Advantage Actor-Critic (A3C)**



Policy-based DRL

- **Asynchronous Advantage Reinforcement Learning (A3C)**

$$\nabla_\theta \log \pi(a_t|s_t;\theta)\,(R_t - b_t(s_t))$$

the *advantage* of action a_t in state s_t.

R_t is an estimate of $Q^\pi(a_t, s_t)$ and b_t is an estimate of $V^\pi(s_t)$

$$\nabla_{\theta'} \log \pi(a_t|s_t;\theta') A(s_t, a_t; \theta, \theta_v)$$

$$\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k};\theta_v) - V(s_t;\theta_v)$$

Policy-based DRL

- **Asynchronous Advantage Reinforcement Learning (A3C)**

Algorithm S2 Asynchronous advantage actor-critic - pseudocode for each actor-learner thread.

// Assume global shared parameter vectors θ and θ_v and global shared counter $T = 0$
// Assume thread-specific parameter vectors θ' and θ'_v
Initialize thread step counter $t \leftarrow 1$
repeat
  Reset gradients: $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.
  Synchronize thread-specific parameters $\theta' = \theta$ and $\theta'_v = \theta_v$
  $t_{start} = t$
  Get state s_t
  repeat
    Perform a_t according to policy $\pi(a_t|s_t;\theta')$
    Receive reward r_t and new state s_{t+1}
    $t \leftarrow t + 1$
    $T \leftarrow T + 1$
  until terminal s_t **or** $t - t_{start} == t_{max}$
  $R = \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t, \theta'_v) & \text{for non-terminal } s_t \text{// Bootstrap from last state} \end{cases}$
  for $i \in \{t-1, \ldots, t_{start}\}$ **do**
    $R \leftarrow r_i + \gamma R$
    Accumulate gradients wrt θ': $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i;\theta')(R - V(s_i;\theta'_v))$
    Accumulate gradients wrt θ'_v: $d\theta_v \leftarrow d\theta_v + \partial \left(R - V(s_i;\theta'_v)\right)^2 / \partial \theta'_v$
  end for
  Perform asynchronous update of θ using $d\theta$ and of θ_v using $d\theta_v$.
until $T > T_{max}$

Policy-based DRL

- **Deep Deterministic Policy Gradient (DDPG)**
  - **Continuous action space**
  - **Deterministic action** $a_t = \mu(s_t)$

Actor-critic + **DQN**

- Policy net + Value net
- Memory Replay
- Main net & Target net

Policy : μ Policy :μ'
Value: Q Value: Q'

Target Q: $Q'\left(s_{i+1}, \mu'\left(s_{i+1}\middle|\theta^{\mu'}\right)\middle|\theta^{Q'}\right)$

Evaluation Q: $Q(s_i, a_i|\ \theta^Q)$

Target reward y_i: $y_i = r_i + \gamma Q'\left(s_{i+1}, \mu'\left(s_{i+1}\middle|\theta^{\mu'}\right)\middle|\theta^{Q'}\right)$

Loss:

$L = \frac{1}{N}\sum i(y_i - Q(s_i,\ a_i|\ \theta^Q)\ )^2$

$L = \frac{1}{N}\sum i(r_i + \gamma Q'\left(s_{i+1}, \mu'\left(s_{i+1}\middle|\theta^{\mu'}\right)\middle|\theta^{Q'}\right) - Q(s_i,\ a_i|\ \theta^Q)\ )^2$

Policy-based DRL

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

BIT.MLMR

End of Lecture 8

Have you grasped the main ideas behind the agents and reinforcement learning?

LIUXIABI@BIT.EDU.CN





